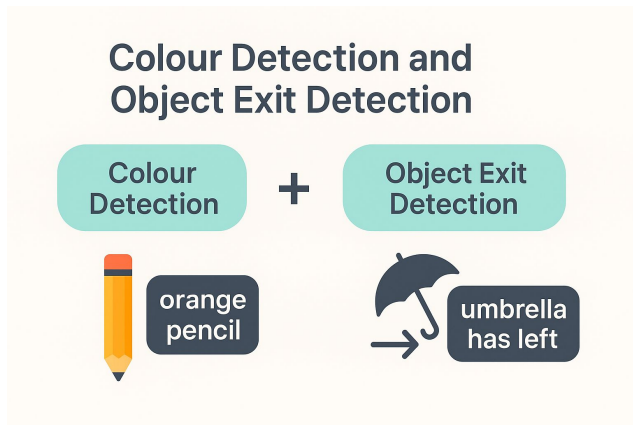


Object Detection

- Color Detection
- Logo Detection

Added 2 Features - Colour Detection and Object Exit Detection





Real-Time Object Detection - with Color and Exit Voice Feedback

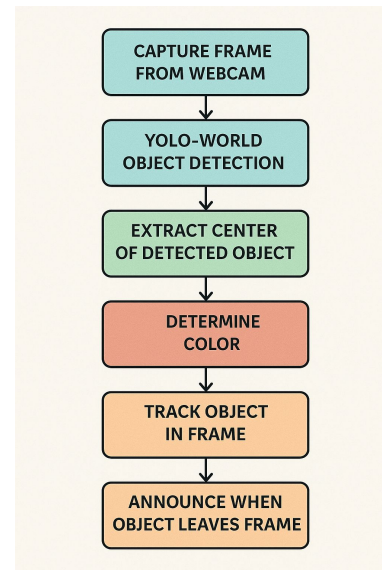
This Python script integrates **YOLO-World** object detection with **real-time voice-based feedback**, providing both:

1. **Accurate color identification** of detected objects.
2. **Spoken alerts when objects leave the camera frame.**

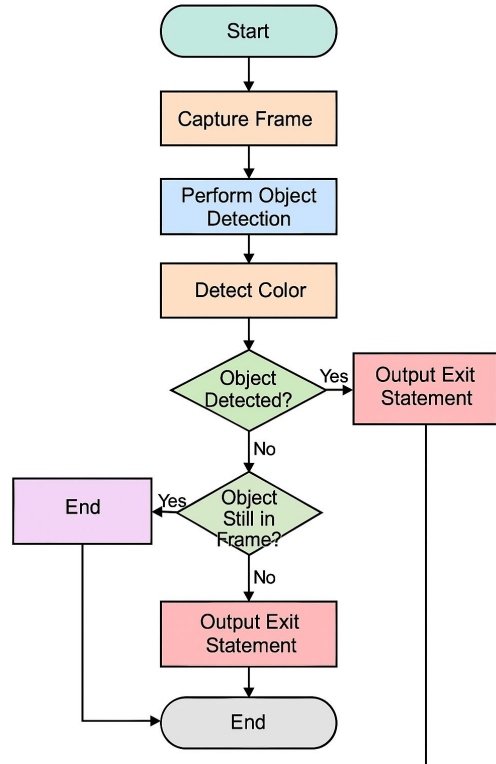


What Was Solved

- Fixed a bug where **all colors were detected as gray** due to averaging noisy bounding boxes.
- Implemented a reliable mechanism to **track disappearing objects** and avoid false alerts.



Flow of how to code works

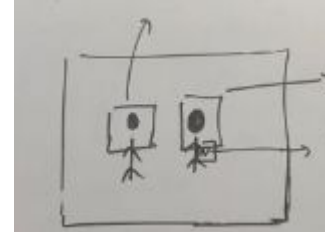


Feature - Colour detection



💡 “ This is a blue phone”

Feature - Exit detection



💡 “ person has left the frame”

Key Features:

Color Detection (Improved Accuracy)

- Instead of averaging the full bounding box (which often captures background noise), the system extracts a small **10×10 pixel region at the center of the detected object**.
- This central patch better represents the object's true color.
- The color is converted from **BGR to HSV** and mapped to a named color (e.g., blue, red, black).

Voice Output with pyttsx3

- Announces object name and color, e.g., *"This is a blue phone"*.
- Prevents repetition by storing a set of already spoken labels.

Exit Detection (Object Left Frame)

- If an object is not detected for **10 consecutive frames**, it announces: *"Umbrella has left the frame."*



Technologies Used:

YOLOWorld (Hugging Face inference API)

OpenCV (video processing)

pyttsx3 (offline voice synthesis)

NumPy (color averaging) - **New one**

Supervision (annotation and bounding box utilities)

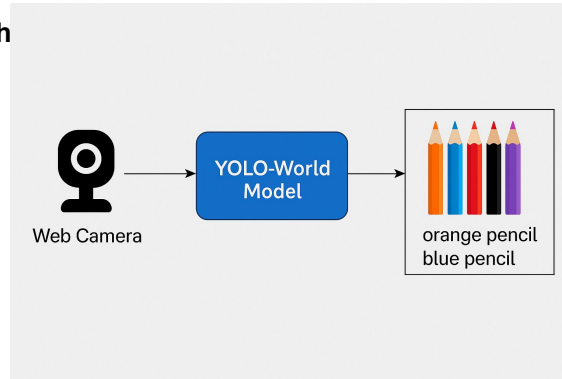
Experiment - to check if all the colour works

🎯 Objective:

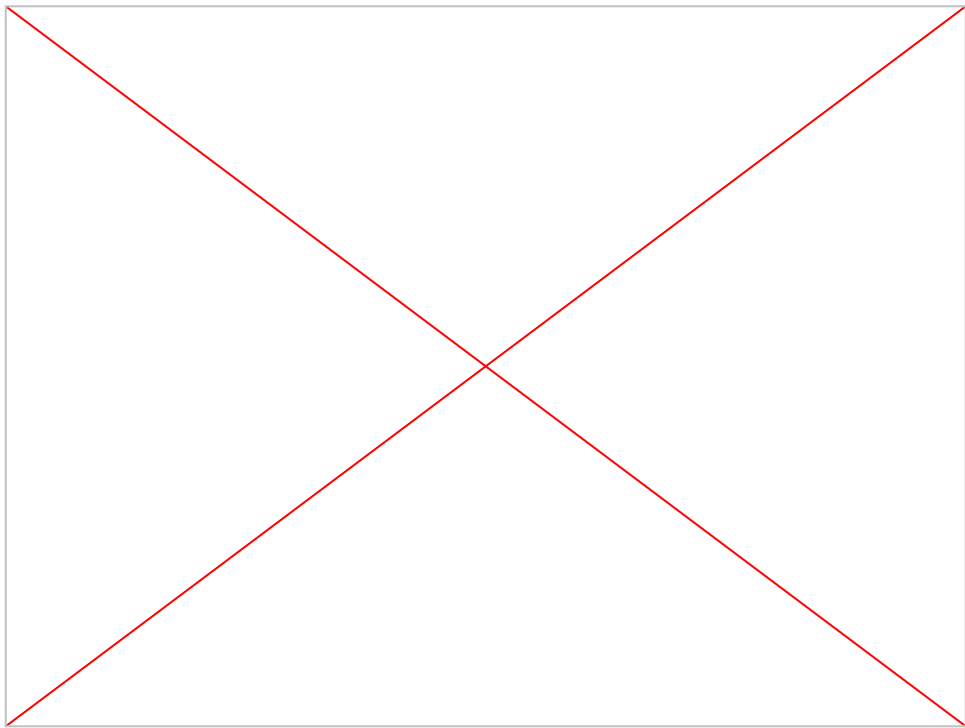
Test whether the real-time detection model can correctly identify and vocalize colors using the improved **center-patch HSV logic**.

🔍 Method:

1. Each pencil was shown to the webcam individually with minimal background interference.
2. The script used a **10×10 center patch** of the bounding box for precise color extraction.
3. The model inferred the object and color.
4. The voice system announced each detected color.



Experiment Results in Real time camera



Objects that can be tested

"person", "woman in white dress", "sunglasses", "hat", "scarf", "gloves",
"watch", "necklace", "umbrella", "clothes hanger", "shopping bag", "cup",
"notepad", "pencil", "pen", "phone", "remote control", "TV", "laptop",
"charger", "keyboard", "chair", "table", "sofa", "water bottle", "fork",
"spoon", "bowl", "banana", "apple", "tree", "flower", "plant", "gun",
"book", "towel", "pair of shoes"

```
conda activate yolo-world  
cd %USERPROFILE%\Downloads  
python detect_voice_color_exit_final.py
```



Real time camera Testing for colour and exit features

The screenshot displays a real-time camera testing interface. On the left, a video feed shows a person with a purple bounding box and the label "orange person (0.90)". A black terminal window in the center shows the following text:

```
...final.py  
...is out of date! Please upgrade to __init__.py:35  
...est features and bug fixes by running  
...Annotator' is deprecated and has been renamed to 'B  
...26.0.
```

Below the terminal, a code editor contains the following commands:

```
conda activate yolo-world  
cd %USERPROFILE%\Downloads  
python detect_voice_color_exit_final.py
```

On the right, a grid of 30 icons represents various objects for classification, including: person, woman in white dress, sunglasses, hat, scarf, bag, cup, watch, necklace, clothes hanger, shopping bag, phone, remote con, TV, charger, keyboard, water, fork, spoon, banana, apple, tree, table, towel, plant, plant, gun, book, towel, a pair of shoes, and of shoes.

The interface also includes a sidebar with a list of slides (1-8) and a bottom status bar showing "33°C Partly sunny" and system icons.

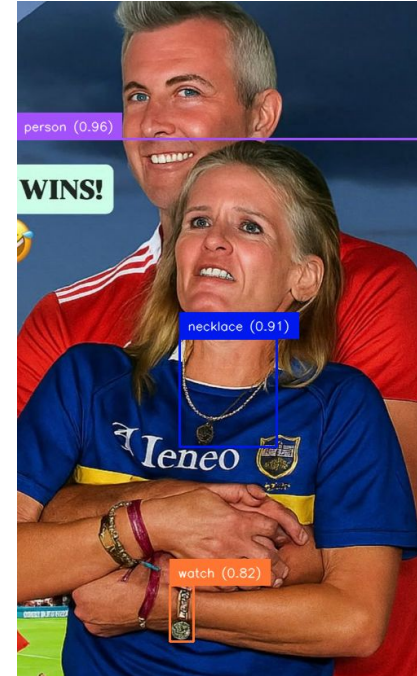
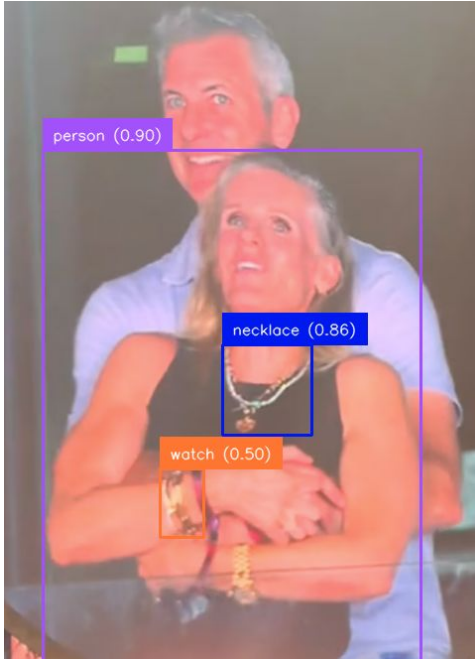
Glitch / Challenges / Observations to overcome

1. Some **objects colours** are detected wrong.
2. Only with **white background** it looks accurate.
3. **Lag** and very slow

Future Scope

1. Trying to overcome **these issues**.
2. Estimate the **distance** of the object from the camera
3. Object left or right (the **directions**)
4. The 10 frames detection can be reduced

Andy Byron's fun Object Detection



Logo Detection using YOLOv8 with Flickr Image Pipeline

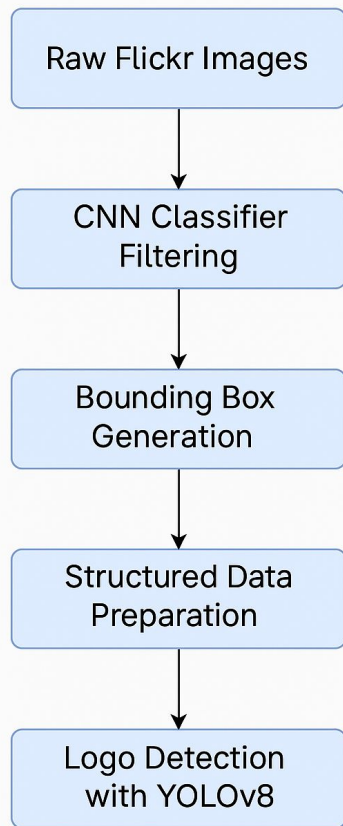


Objective

- Automatically detect logos using YOLOv8 in real-time webcam feed.
- Build the dataset using scraped images from Flickr.
- Convert classification-style logo folders into object detection format.



Workflow Overview



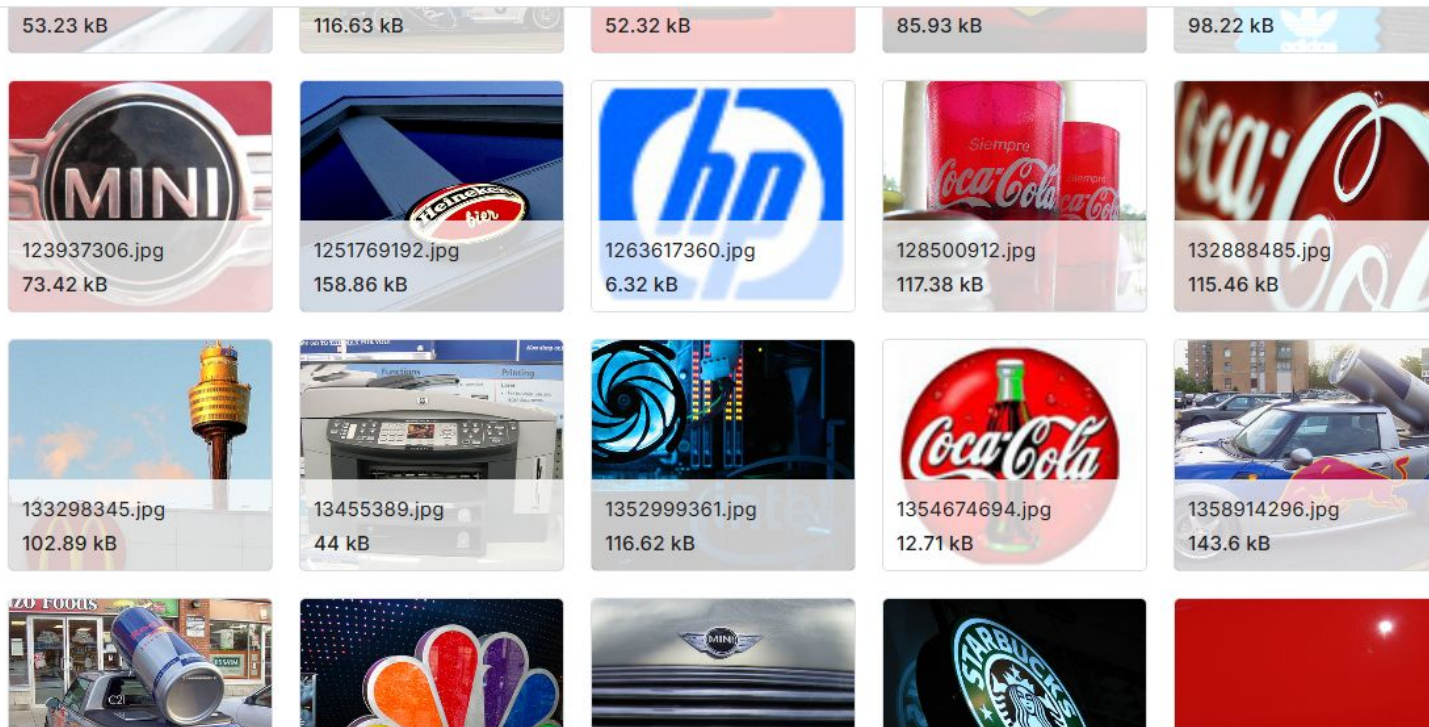
[Logo Classes,](#)
[Labels and](#)
[YOLO](#)
[Formatting](#)

```
[ ] trainset.class_indices  
  
↔ { 'Adidas': 0,  
    'Apple': 1,  
    'BMW': 2,  
    'Citroen': 3,  
    'Cocacola': 4,  
    'DHL': 5,  
    'Fedex': 6,  
    'Ferrari': 7,  
    'Ford': 8,  
    'Google': 9,  
    'HP': 10,  
    'Heineken': 11,  
    'Intel': 12,  
    'McDonalds': 13,  
    'Mini': 14,  
    'Nbc': 15,  
    'Nike': 16,  
    'Pepsi': 17,  
    'Porsche': 18,  
    'Puma': 19,  
    'RedBull': 20,  
    'Sprite': 21,  
    'Starbucks': 22,  
    'Texaco': 23,  
    'Unicef': 24,  
    'Vodafone': 25,  
    'Yahoo': 26 }
```

Xception! - Flickr27 : Brand Logo Detection

Notebook Input Output Logs Comments (4)

flickr_logos_27_dataset_images (1079 files)



Manual Work Beyond Flickr for YOLOv8 Training

Flickr Dataset Limitations

- Raw keyword-based image scraping only
- No labels or bounding boxes provided

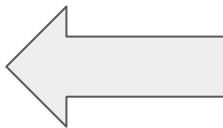


Manual Steps Performed

- **CNN Classifier Filtering:**
Trained a model to remove irrelevant images from Flickr results
- **Logo Folder Sorting:**
Verified class-wise folders for logo accuracy
- **Bounding Box Generation:**
Created pseudo YOLO annotations with centered fixed-size bounding boxes
- **Dataset Structuring:**
Prepared `images/` and `labels/` folders with YOLO format
Authored `data.yaml` manually to define paths and class names

Outcome

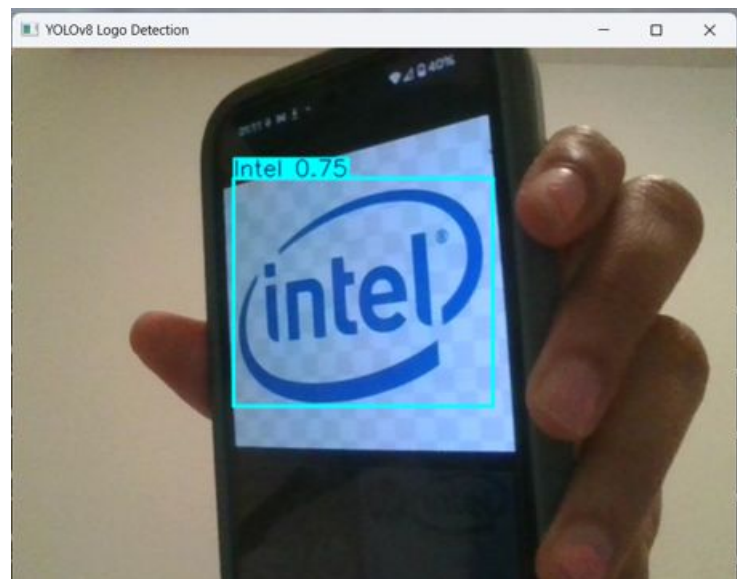
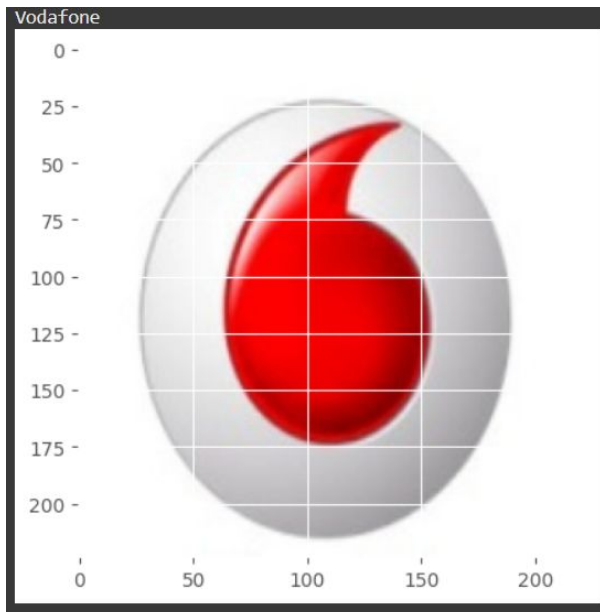
- Transformed raw Flickr data into a usable YOLOv8 training dataset
- Enabled high-accuracy logo detection in real-time via webcam

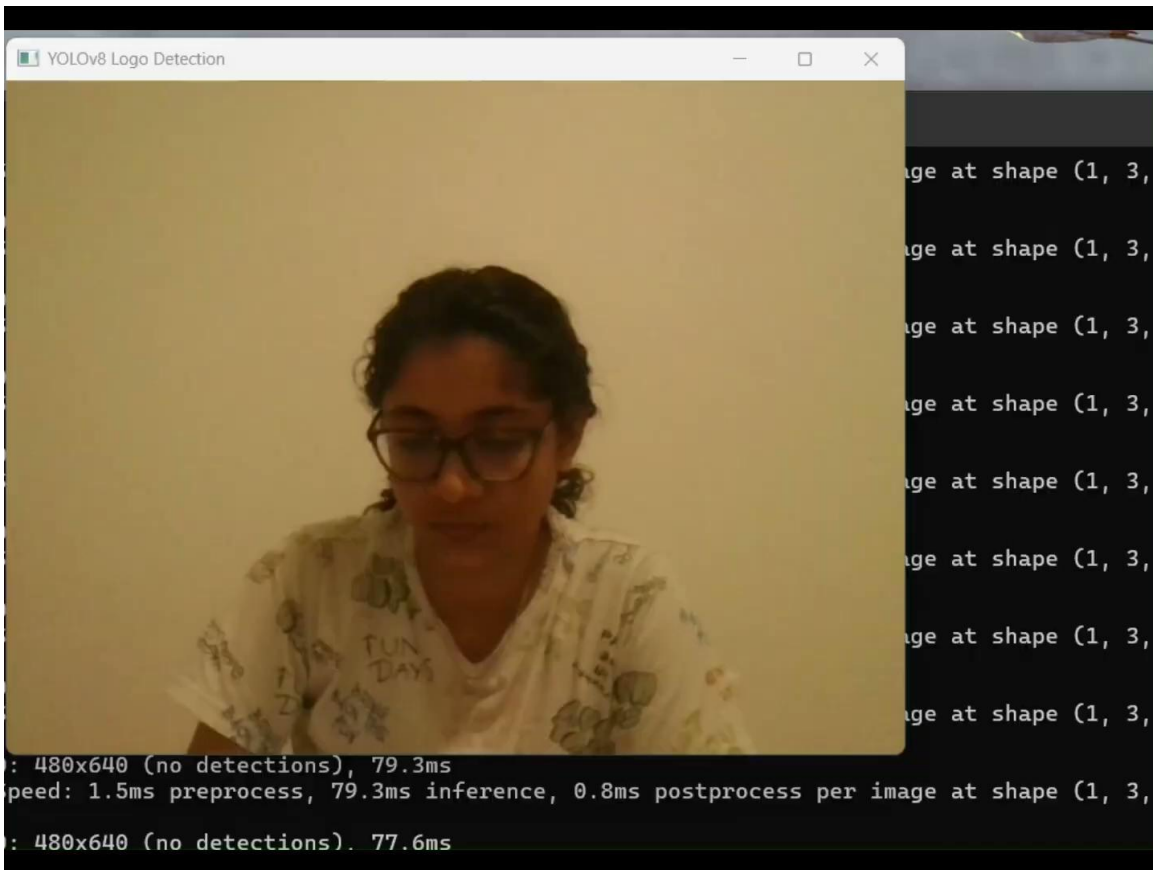


Summary:

Data Analysis Key Findings

- 4536 annotations were extracted from the training set annotation file.
- 27 unique class labels were identified and mapped to numerical IDs from 0 to 26.
- Bounding box coordinates were successfully converted from the original format (x1, y1, x2, y2) to the YOLO format (center_x, center_y, width, height relative to image dimensions) normalized by image dimensions.







Future Improvements

- **Expand Logo Classes**
→ Gradually increase from 27 to 100+ logos while maintaining class balance.
- **Enhance Bounding Box Precision**
→ Refine manual annotations for better object localization.
- **Include Challenging Scenarios**
→ Add images with cluttered backgrounds, occlusions, distortions, and real-world settings.
- **Automate Annotation**
→ Use semi-supervised tools (e.g., Label Studio with auto-suggest) to speed up labeling.
- **Multilingual or Brand Variants**
→ Include regional variants of logos and test cross-brand generalization.
- **Feedback Loop from False Detections**
→ Collect incorrect detections and retrain to improve robustness.

